

Patient-First Interoperable Patient-Provider Portal: User Guide

CS-6440 Team Platypus (Group 002)

November 23, 2025

Accessing the Application

Live Deployment

The PLATYPUS Patient-Provider Portal is deployed and accessible at:

`https://platypus-htck.onrender.com`

No installation or setup is required to use the live application. Simply navigate to the URL above in your web browser.

User Authentication

Patient Login Credentials

The following patient accounts are available for testing:

- **Username:** abechtel **Password:** Pat1
- **Username:** dfahley **Password:** Pat2
- **Username:** mwaters **Password:** Pat3
- **Username:** phalvors **Password:** Pat4

Provider Login Credentials

The following provider accounts are available for testing:

- **Username:** akeeling **Password:** Pra1
- **Username:** ckoch **Password:** Pra2

Using the Application

Login Process

1. Navigate to **`https://platypus-htck.onrender.com`**
2. Click the **login** button on the main page
3. Enter your username and password in the popup dialog
4. Click **Login** to authenticate
5. You will be automatically redirected based on your user type:
 - Patients are redirected to their personal chart view
 - Providers are redirected to their dashboard

If login credentials are incorrect, an error message will be displayed. Users attempting to access protected pages without authentication will be automatically redirected to the login page.

Patient Features

Personal Health Chart

After logging in, patients can view their comprehensive health information organized into collapsible sections:

- **Demographic Information:** Name, date of birth, gender, address, and phone number
- **Health Conditions:** Current and past medical conditions with onset dates
- **Vaccinations:** Complete vaccination history with dates and locations
- **Past Encounters:** Medical visits, appointments, and procedures with provider and location details
- **Recent Observations:** Latest lab results, vital signs, and other clinical measurements

Chart Navigation

- Use the **Expand All / Collapse All** button to show or hide all sections at once
- Click on individual section headers to expand or collapse specific sections
- The Demographics section is expanded by default for quick access to basic information
- If an item in a section was commented on by a provider, it will have a green lightbulb icon next to the item name.
- Clicking the **lightbulb** icon for an item will show the provider(s)' comments. Click outside of the popup to close

Accept/Deny Provider Requests

A provider can send a request to a patient to receive access to a restricted section set by the patient. Near the bottom of the patient chart, the patient can accept/deny medical requests from the provider

- Navigate to the **Access Requests** section
- A provider request contains: **Provider Name, Requested Sections, Reason, Date Requested**
- For each request, the patient can either click on the **Allow Access** or **Deny** button
- If the patient selects **Allow Access**, the provider will be able to see the new sections they requested in the provider view.
- If the patient selects **Deny**, the section the provider requested will continue to be locked.

Share Medical Information

At the bottom of the patient health chart, the patient can view their current sharing settings of all the providers that have access to the patient's information

- Expand the **Sharing Settings Section**
- Click on each of the checkboxes to enable/disable access to specific medical sections for the provider
- **The patient can enable/disable:** Demographics, Conditions, Medications, Encounters, Lab Results, Vaccinations

- Click on the **Save Button** to save the changes. The changes apply automatically in the provider view
- To remove a provider, click on the **Remove Provider Button**
- A confirmation dialog will popup to confirm the changes
- If removed, the provider will no longer have access to the patient's medical information and cannot add the patient without an access code
- To add a provider, enter the provider ID in the input field and click on **Find Provider**
- If the provider was found, select the medical sections for the provider to have access to
- Click on **Save and Generate Code** to finish adding the provider
- A new access code is automatically generated for the provider to use to add the patient to their list

Logout Process

1. Click the person button in the bottom toolbar (farthest right)
2. Click the **Logout** button that shows below where the person button was clicked
3. You will be logged out and directed to the login page
4. If the silhouette button is accidentally clicked, click away from the Logout button to hide it

Provider Features

Navigation

The provider interface includes a left navigation panel with the following sections:

- **Patients:** Main patient management dashboard
- **Schedule:** Appointment and scheduling management

Patient Management Dashboard

Providers have access to a comprehensive dashboard with the following features:

- **Patient List:** View all patients currently under their care
- **Patient Search:** Search for and add new patients to their practice
- **Patient Data Access:** View detailed patient information for authorized patients

Adding Patients to Practice

1. Click **Search for Patients** button on the dashboard
2. Enter the patient's ID in the search field
3. Click the search icon to verify the patient exists
4. The patient's name will be displayed for confirmation
5. Enter the required access code in the provided field
6. Click **Add Patient** to complete the process

If the access code is correct, the patient will be added to the provider's patient list and the page will refresh. Incorrect access codes will display an error message.

Viewing Patient Data

1. From the Patient tab on the provider dashboard

2. Locate the desired patient in the patient list table
3. Click the **View Data** button (open icon) in the corresponding row
4. Access the patient's complete medical information based on patient's sharing permissions

Requesting Patient Information Access

A provider can request access to a restricted section of a patient's medical record.

1. Follow the Viewing Patient Data steps to navigate to the patient page
2. Select on **Request Access** on a locked section
3. Confirm the selection and write a reason why the provider needs access
4. Click on the **Send Request** button for the patient to receive the request
5. Click on the **Access Requested** button to view the status of the request
6. Once the patient accepts the request, the provider will be able to view the requested section.
7. If the patient denies the request, the provider will continue to not see the restricted sections.

Adding/Editing Comments on Patient Data

1. While viewing patient data, the provider can double click on an item name within a section to add a comment
2. Type the comment in the text box
3. Click **Save Comment** to save the comment
4. Existing comments will be indicated by a lightbulb icon next to the item name.
5. To edit an existing comment, click the **lightbulb** icon
6. Click outside of the popup to close without saving the comment

Chatting with a Patient

1. From the Patient tab on the provider dashboard
2. Locate the desired patient in the patient list table
3. Click the **Chat** button (chat bubble icon) in the corresponding row
4. The chat page will open with a history of the conversation
5. To view prior chat history, scroll up to view older messages
6. To write a new message, type in the text box and click the **Send** button (paper airplane icon).
7. The page will reload to show the latest chat message in the conversation

Logout Process

1. Click the person button in the top toolbar (farthest right)
2. Click the **Logout** button that shows below where the person button was clicked
3. You will be logged out and directed to the login page
4. If the silhouette button is accidentally clicked, click away from the Logout button to hide it

Technical Features

FHIR Compliance

The application is built on FHIR (Fast Healthcare Interoperability Resources) standards, ensuring:

- Standardized healthcare data exchange

- Interoperability with other healthcare systems
- Secure and structured patient data management

Security Features

- Session-based authentication
- Role-based access control (Patient vs. Provider)
- Access code verification for provider-patient relationships
- Automatic session management and logout

Responsive Design

The application is designed to work across different devices and screen sizes, providing a consistent user experience on desktop, tablet, and mobile devices.

Local Development Setup

For developers who want to run the application locally:

Dependencies

- Python 3.9+ (`python3 --version`)
- FastAPI & Uvicorn:

```
pip3 install "uvicorn[standard]" fastapi
pip3 install pytest
pip3 install -r requirements.txt
```

Load the App

From the repository root (where `backend/` lives):

```
python3 -m uvicorn backend.backend:app --reload
```

Optional Utilities

- `jq` pretty-prints JSON: (run examples without `JQ` if not installed)

```
brew install jq
```

Run the Application Through Docker

The frontend will be at `http://localhost:5173/`.

- If initially setting up or after installing a new npm package: From the repository root, run the following:

```
npm install (package here) --save
docker-compose down -v
docker-compose build --no-cache
docker-compose up
```

- Otherwise: From the repository root, run the following:

```
docker compose up
```

API Documentation

For developers who need to interact with the backend API directly, the following endpoints are available:

Test Patient Endpoints

GET/patient/{id}/demo Returns basic demographics (name, gender, birthDate, telecom, address, identifier).

```
curl -s "http://localhost:8000/patient/38cad98b-4c17-10a1-2052-ba1ce5f10142/demo" | jq
```

GET/patient/{id}/vaccinations Returns Vaccination resources (id, code, name, occurrenceDate-Time, Location).

```
curl -s "http://localhost:8000/patient/709f534c-95c7-31c0-a457-92a7ae1b925b/vaccinations" | jq
```

GET/patient/{id}/medications Returns Medication resources (id, code, name, status).

```
curl -s "http://localhost:8000/patient/709f534c-95c7-31c0-a457-92a7ae1b925b/medications" | jq
```

GET/patient/{id}/conditions Returns Condition resources (id, code, clinicalStatus, verification-Status, onset, recordedDate).

```
curl -s "http://localhost:8000/patient/709f534c-95c7-31c0-a457-92a7ae1b925b/conditions" | jq
```

GET/patient/{id}/encounters Returns Encounter resources (status, class, type, period, service-Provider, location, subject).

```
curl -s "http://localhost:8000/patient/34e8cba5-9a59-6e62-c550-203c23ed3a76/encounters" | jq
```

GET/patient/{id}/observations[?category=...] Returns Observation resources; optional **category** filter (e.g., vital-signs, laboratory, survey).

```
# all observations
curl -s "http://localhost:8000/patient/709f534c-95c7-31c0-a457-92a7ae1b925b/observations" | jq

# filter by category
curl -s "http://localhost:8000/patient/709f534c-95c7-31c0-a457-92a7ae1b925b/observations?category=survey" | jq
```

POST/patient/batch Returns any set of Patient FHIR resources based on the API body parameters passed.

```
curl -X POST "http://localhost:8000/patient/batch" \
-H "Content-Type: application/json" \
-d '{
  "demo": { "id": "34e8cba5-9a59-6e62-c550-203c23ed3a76" },
  "observations": { "id": "34e8cba5-9a59-6e62-c550-203c23ed3a76", "category": "
Laboratory" },
  "medications": { "id": "34e8cba5-9a59-6e62-c550-203c23ed3a76" },
  "conditions": { "id": "34e8cba5-9a59-6e62-c550-203c23ed3a76" },
  "encounters": { "id": "34e8cba5-9a59-6e62-c550-203c23ed3a76" },
  "vaccinations": { "id": "34e8cba5-9a59-6e62-c550-203c23ed3a76" }
}' | jq
```

Test Provider Endpoints

GET/providers[?specialty=...] Lists PractitionerRole joined with Practitioner (NPI, name, gender, telecom, organization, location). Optional specialty code filter.

```
curl -s "http://localhost:8000/providers" | jq
curl -s "http://localhost:8000/providers?specialty=208D00000X" | jq
```

GET/providers/{npi} Returns a single provider by NPI.

```
curl -s "http://localhost:8000/providers/9999984591" | jq
```

GET/clinics Lists Organizations and Locations.

```
curl -s "http://localhost:8000/clinics" | jq
```

POST/provider/add Returns a patient's name if a provider has an access code for them. Saves their id under that provider's list of patients

```
curl -X GET "http://localhost:8000/provider/add" \
-H "Content-Type: application/json" \
-d '{
  "patientID": "34e8cba5-9a59-6e62-c550-203c23ed3a76",
  "providerID": "e7300854-560e-f9a6-6207-4e4a57123acf",
  "code": "208D00000X"
}' | jq
```

GET/provider/patients/{id} Returns the patient list for a particular provider

```
curl -s "http://localhost:8000/provider/patients/e7300854-560e-f9a6-6207-4e4a57123acf" |
jq
```

POST/provider/getpatient Returns the patient's data that the provider has access to

```
curl -X GET "http://localhost:8000/provider/add" \
-H "Content-Type: application/json" \
-d '{
  "patientID": "34e8cba5-9a59-6e62-c550-203c23ed3a76",
  "providerID": "e7300854-560e-f9a6-6207-4e4a57123acf"
}' | jq
```

GET/provider/name/{id} Returns the provider's display name

```
curl -s "http://localhost:8000/provider/name/e7300854-560e-f9a6-6207-4e4a57123acf" | jq
```

POST/comments/provider Returns all comments from a provider on a patient's chart

```
curl -X GET "http://localhost:8000/comments/provider" \  
-H "Content-Type: application/json" \  
-d '{  
  "patientID": "38cad98b-4c17-10a1-2052-ba1ce5f10142",  
  "providerID": "e7300854-560e-f9a6-6207-4e4a57123acf"  
}' | jq
```

POST/comments/add Adds a comment to the patient's data from the provider

```
curl -X GET "http://localhost:8000/comments/provider" \  
-H "Content-Type: application/json" \  
-d '{  
  "patientID": "38cad98b-4c17-10a1-2052-ba1ce5f10142",  
  "providerID": "8abea618-a9d0-5088-bf60-c8dd0f6a4253",  
  "resourceID": "38cad98b-4c17-10a1-6b1a-3c2219999652",  
  "comment" : "Let's reevaluate this medication again at your next appointment"  
}' | jq
```

Test Chat Endpoints

POST/chat/history Returns the chat history between the corresponding patient and provider

```
curl -X GET "http://localhost:8000/chat/history" \  
-H "Content-Type: application/json" \  
-d '{  
  "patientID": "34e8cba5-9a59-6e62-c550-203c23ed3a76",  
  "providerID": "e7300854-560e-f9a6-6207-4e4a57123acf"  
}' | jq
```

POST/chat/send Adds a new message to the chat history between the corresponding patient and provider

```
curl -X GET "http://localhost:8000/chat/send" \  
-H "Content-Type: application/json" \  
-d '{  
  "patientID": "34e8cba5-9a59-6e62-c550-203c23ed3a76",  
  "providerID": "e7300854-560e-f9a6-6207-4e4a57123acf",  
  "senderID": "e7300854-560e-f9a6-6207-4e4a57123acf",  
  "message": "this is a test message"  
}' | jq
```

Test Request Medical Information Access Endpoints

POST(/access/request) Create a new access request from provider to patient


```
curl -X POST "http://localhost:8000/access/request" \
-H "Content-Type: application/json" \
-d '{
  "patientID": "38cad98b-4c17-10a1-2052-ba1ce5f10142",
  "providerID": "e7300854-560e-f9a6-6207-4e4a57123acf",
  "sections": ["observations:Laboratory"],
  "reason": "Need lab results for treatment planning"
}' | jq
```

GET/access/requests/provider/provider_id Get all access requests for a provider

```
curl -X GET "http://localhost:8000/access/requests/provider/e7300854-560e-f9a6-6207-4e4a57123acf" \
-H "Content-Type: application/json" | jq
```

GET/access/requests/patient/patient_id Gets all the provider access requests for a patient

```
curl -X GET "http://localhost:8000/access/requests/patient/38cad98b-4c17-10a1-2052-ba1ce5f10142" \
-H "Content-Type: application/json" | jq
```

POST/access/requests/request_id/respond Patient responds to a provider access request (accept/deny)

```
curl -X POST "http://localhost:8000/access/requests/1a92008d-e5b7-49d6-bd74-02a4218a6031/respond" \
-H "Content-Type: application/json" \
-d '{
  "requestID": "1a92008d-e5b7-49d6-bd74-02a4218a6031",
  "status": "accepted"
}'
```

Test Share Medical Information Endpoints

GET/patient/providers/patient_id Returns all providers a patient is currently sharing with

```
curl -X GET "http://localhost:8000/patient/providers/38cad98b-4c17-10a1-2052-ba1ce5f10142" \
-H "Content-Type: application/json" | jq
```

POST/patient/share Updates the specific medical chart sections a specific provider can see

```
curl -X POST "http://localhost:8000/patient/share" \
-H "Content-Type: application/json" \
-d '{
  "patientID": "38cad98b-4c17-10a1-2052-ba1ce5f10142",
  "providerID": "e7300854-560e-f9a6-6207-4e4a57123acf",
  "sections": ["demographics", "conditions", "encounters"]
}' | jq
```

POST/patient/remove_provider Removes a provider's access to a patient's chart

```
curl -X POST "http://localhost:8000/patient/remove_provider" \
-H "Content-Type: application/json" \
-d '{
    "patientID": "38cad98b-4c17-10a1-2052-ba1ce5f10142",
    "providerID": "e7300854-560e-f9a6-6207-4e4a57123acf"
}' | jq
```

Running API Endpoints Unit Tests

The unit tests are ran using pytest command. The unit tests do not require Docker to be turned on. From the repository root (where `backend/` lives):

```
pytest -v
```

Development Notes

Resetting the Application After A Demo

To keep the demo consistent, the changes during a demo can be reversed using the `demo_reset.py` in the backend folder. To reset, include the necessary code to overwrite the data files and undo changes. Then run “`python demo_reset.py`” in the backend folder to reset the data for the application.